

Министерство образования Московской области
ГБПОУ МО «Серпуховский колледж»

ОТЧЕТ ПО ПРОИЗВОДСТВЕННОЙ ПРАКТИКЕ
по ПМ.01 «РАЗРАБОТКА МОДУЛЕЙ ПРОГРАММНОГО
ОБЕСПЕЧЕНИЯ ДЛЯ КОМПЬЮТЕРНЫХ СИСТЕМ»

Специальность
09.02.07 Информационные системы и программирование
квалификация Программист

Руководитель практики
преподаватель Плетнева Г.Н.
«__» _____2026г.

(подпись)

Исполнитель
Студент Починский Г.Д.,
курс 3, группа 1231
«__» _____2026г.

(подпись)

Серпухов 2026 г.

СОДЕРЖАНИЕ

Введение	3
Постановка задачи	4
Аналитическая часть	5
Проектная часть	8
Заключение.....	23
Список источников	24
ПРИЛОЖЕНИЕ	25

Введение

1.1. О предприятии (организации):

1.1.1. Практика проходила на ООО «РАТЕП-ИННОВАЦИЯ». Предприятие занимается строением, разработкой, починкой и отладкой систем навигации, бортовых компьютеров, системных блоков и много чем ещё для паровозов и электричек;

1.1.2. Практика проходила в информационном отделе. Основная задача информационного отдела - облегчение и помощь в автоматизации других отделов, например документооборот в бухгалтерии;

1.2. Цель работы:

1.2.1. Цель работы - создание мобильного приложения для просмотра, редактирования и работы с заказами предприятия. Приложение предназначено для автоматизации и упрощения отслеживания заказов, их статуса и срока заказа;

1.3. Актуальность:

1.3.1. Приложение поможет оперативно отслеживать заказы без подходов к компьютерам или проверки списков на бумаге. Приложение обеспечит быстрый онлайн доступ к заказам, при том с общей БД, что позволит отслеживать заказы в реальном времени. Ожидаемый результат – сокращение времени на поиск и выполнение заказов.

Постановка задачи

2.1. Краткое описание задания:

2.1.1. В рамках задания необходимо создать мобильное приложение на платформе Android, предметной областью которого является – учет заказов предприятия.

2.1.2. Основное назначение приложения – внедрение каталога заказов с разделением на статусы;

2.1.3. Платформа разработки: Android

2.1.4. Используемые технологии и инструменты:

Язык: Kotlin.

Среда разработки: Android Studio.

Локальная БД: SQLite

Доп. библиотеки: Glide (загрузка фото)

Общий перечень требований:

Наличие авторизации и регистрации;

Работа с основными объектами предметной области (создание, просмотр, редактирование),

Хранение данных,

Рабочие фильтрация и поиск,

Требования к интерфейсу и пользовательскому опыту.

Аналитическая часть

3.1. Анализ существующих решений

Для определения требований к разрабатываемому приложению были проанализированы существующие мобильные решения для просмотра и взаимодействия с производственными заказами.

Обзор аналогов:

- «1С:Мобильный рабочий» — стандартный клиент для учёта задач и документов в экосистеме 1С.
- «SAP Production Order Mobile» — корпоративное решение для управления заказами на крупных предприятиях.
- «Битрикс24 / Задачи» — универсальный трекер цеха, который иногда адаптируют под учёт нарядов.

Таблица №1 – Сравнительная таблица мобильных приложений

Критерий	1С:Мобильный рабочий	SAP Production Order Mobile	Битрикс24 / Задачи	РАТЕП-ЗАДАЧИ
Функциональность	Узкоспециализированная	Избыточная	Базовая	Оптимальная
Удобство интерфейса	Среднее	Перегружен	Простой	Современный
Доступность	Требуется сервер	Частичный офлайн	Онлайн	Полный офлайн
Цена	Платно	Высокая	Бесплатно/Платно	Бесплатно
Разделение ролей (админ/пользователь)	Да	Да	Частично	Да

Проанализированные аналоги имеют существенные недостатки применительно к задачам цехового контроля:

- Интерфейс либо перегружен смежными модулями (логистика, финансы, HR), либо слишком упрощён для производственного контекста.
- Отсутствует удобный механизм черновиков согласования и быстрого обновления статусов заказов прямо на линии.

Уникальные особенности разработанного приложения:

- Просмотр заказа с возможностью его редакции;
- Оптимальный дизайн;
- Роли «пользователь» и «администратор»;

3.2. Разработка стиль-гайда

Для обеспечения единого вида приложения был разработан стиль-гайд. Стиль-гайд определяет цветовую палитру, вид и форму UI элементов. Основные положения представлены на рисунке 1.



Рисунок 1

3.3. Описание целевой аудитории

Целевая группа приложения – рабочие цехов и руководители: Возраст 25-50 лет. Сценарий: быстро найти невыполненный заказ и брать его в работу или сдать выполненный заказ.

Вторичная группа приложения – менеджеры и офисные сотрудники, отвечающие за каталог производимых услуг, оформление заказов и т.д.

3.4. Описание входных и выходных данных

Входные данные: Логин, пароль и ФИО пользователя, Название товара и его статус

Выходные данные: Список заказов

3.5. Требования к программе

Функциональные: Регистрация, CRUD заказов, фильтр по категориям и поиск по названию.

Надежность: Обработка исключений и валидация полей ввода (пустые строки, неправильный формат, длина пароля и т.д.). Приложение не должно падать при пустой БД.

Информативность: Вывод Toast-сообщений об успехе/ошибке.

Проектная часть

4.1. Логическая модель базы данных

Концептуальная модель БД представлена на рисунке 2. Выделенные сущности: «Пользователь», «Заказ», «Товар» со связями типа «один-ко-многим» (1:M)

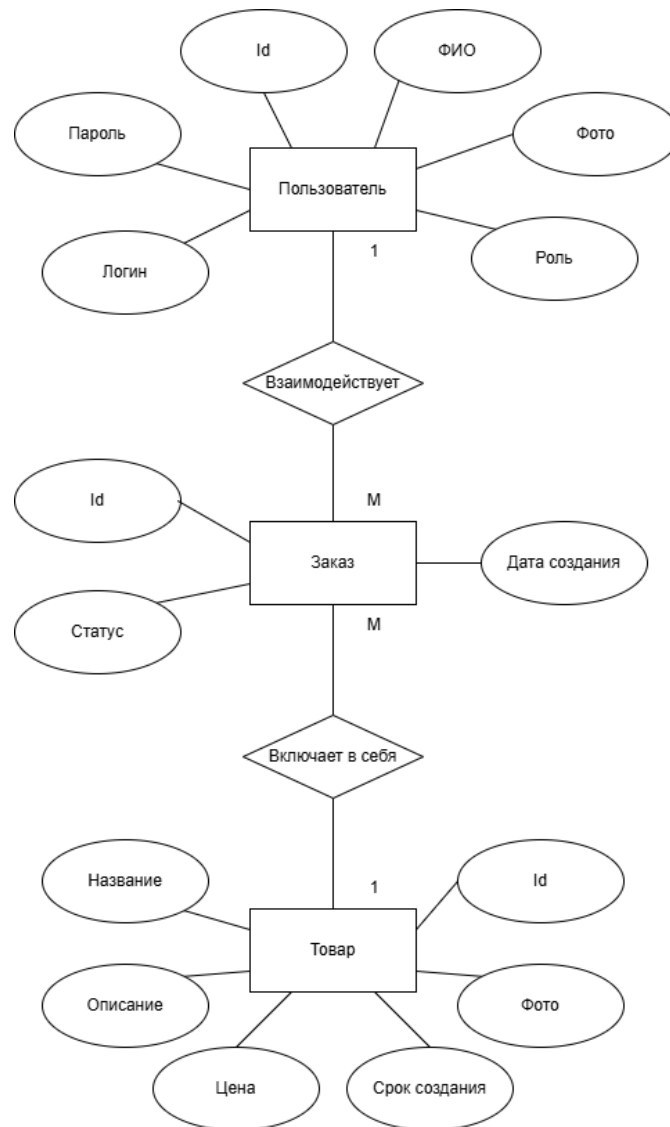


Рисунок 2

4.2. Физическая модель базы данных

Как СУБД выбрана SQLite. Физическая модель данных представлена на рисунке 3.

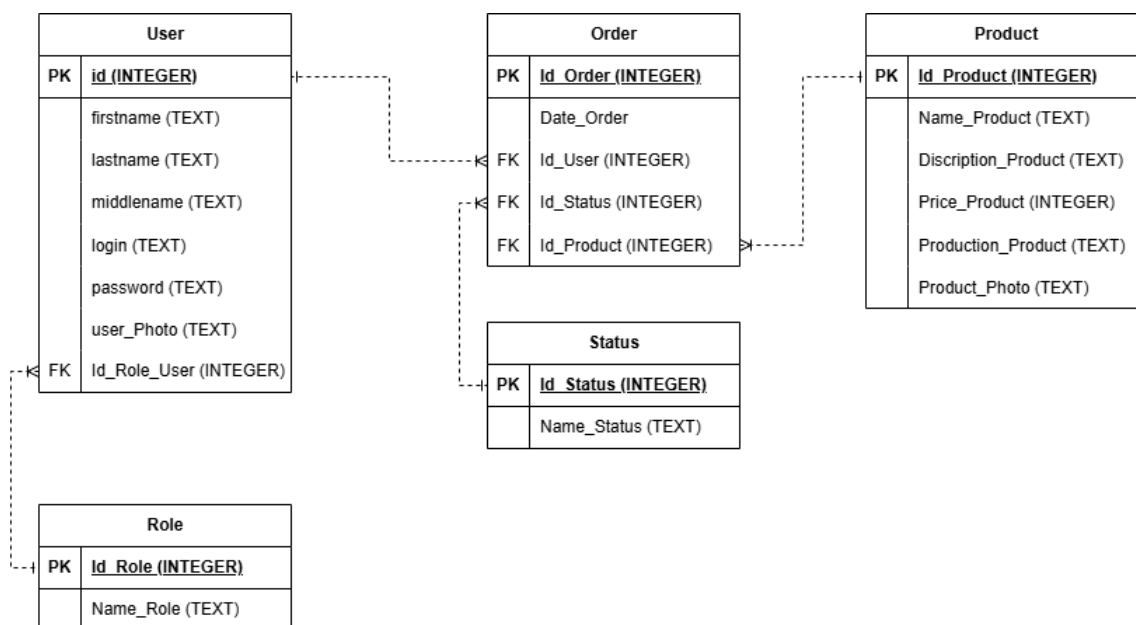


Рисунок 3

4.3. Описание структуры таблиц (Словарь данных)

Детальное описание БД представлена в таблицах 2 – 6

Таблица № 2 – Users

Поле	Тип данных	Ограничения	Описание
id	INTEGER	PK, AUTOINCREMENT	Уникальный идентификатор пользователя
firstname	TEXT		Имя
lastname	TEXT		Фамилия
middlename	TEXT		Отчество
login	TEXT	NOT NULL, UNIQUE	Логин для входа
password	TEXT	NOT NULL	Пароль
user_Photo	TEXT		Путь к фото пользователя
Id_Role_User	INTEGER	FK → Role(Id_Role)	Роль пользователя

Таблица № 3 – Order

Поле	Тип данных	Ограничения	Описание
Id_Order	INTEGER	PK, AUTOINCREMENT	Уникальный идентификатор заказа
Date_Order	TEXT	NOT NULL	Дата оформления заказа
Id_User	INTEGER	FK → Users(id)	Пользователь, оформивший заказ
Id_Status	INTEGER	FK → Status(Id_Status)	Статус заказа
Id_Product	INTEGER	FK → Product(Id_Product)	Товар в заказе

Таблица № 4 – Product

Поле	Тип данных	Ограничения	Описание
Id_Product	INTEGER	PK, AUTOINCREMENT	Уникальный идентификатор товара
Name_Product	TEXT	NOT NULL	Название товара
Discription_Product	TEXT		Описание товара
Price_Product	INTEGER	NOT NULL	Цена товара
Production_Product	TEXT		Производитель
Product_Photo	TEXT		Путь к фото товара

Таблица № 5 – Status

Поле	Тип данных	Ограничения	Описание
Id_Status	INTEGER	PK, AUTOINCREMENT	Уникальный идентификатор статуса
Name_Status	TEXT	NOT NULL, UNIQUE	Название статуса (например, «Новый», «Оплачен», «Доставлен»)

Таблица № 6 – Role

Поле	Тип данных	Ограничения	Описание
Id_Role	INTEGER	PK, AUTOINCREMENT	Уникальный идентификатор роли
Name_Role	TEXT	NOT NULL, UNIQUE	Название роли (например, «Администратор», «Пользователь»)

4.4. SQL-сценарий разработки базы данных

Для создания структуры базы данных используются SQL-запросы на создание таблиц. Полный SQL-сценарий представлен на рисунках 4 – 5

```
CREATE TABLE Role (
    Id_Role INTEGER PRIMARY KEY AUTOINCREMENT,
    Name_Role TEXT
)

CREATE TABLE Status (
    Id_Status INTEGER PRIMARY KEY AUTOINCREMENT,
    Name_Status TEXT
)
```

Рисунок 4

```

CREATE TABLE Users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    firstname TEXT,
    lastname TEXT,
    middlename TEXT,
    login TEXT,
    password TEXT,
    user_Photo TEXT,
    Id_Role_User INTEGER,
    FOREIGN KEY(Id_Role_User) REFERENCES Role(Id_Role)
)

CREATE TABLE Product (
    Id_Product INTEGER PRIMARY KEY AUTOINCREMENT,
    Name_Product TEXT,
    Discription_Product TEXT,
    Price_Product INTEGER,
    Production_Time TEXT,
    Product_Photo TEXT
)

CREATE TABLE "Order" (
    Id_Order INTEGER PRIMARY KEY AUTOINCREMENT,
    Date_Order TEXT,
    Id_Status INTEGER,
    Id_User INTEGER,
    Id_Product INTEGER,
    FOREIGN KEY(Id_Status) REFERENCES Status(Id_Status),
    FOREIGN KEY(Id_User) REFERENCES Users(id),
    FOREIGN KEY(Id_Product) REFERENCES Product(Id_Product)
)

```

Рисунок 5

4.5. Разработка требований к интерфейсу

В приложении были использованы: EditText для ввода информации, TextView для вывода информации, Button для подтверждения действий или навигации, ScrollView и LinerLayout для разлиновки страниц, ImageView для картинок, Spinner для выпадающих списков, RecyclerView для вывода заказов.

4.6. Проектирование макета интерфейса (Figma)

Проектирование пользовательского интерфейса было в соответствии со стиль-гайдом на платформе Figma. Ознакомиться с макетами можно по ссылке:

<https://www.figma.com/design/vcAarRZ5UtUhQSlRNfWxvn/Untitled?node-id=0-1&p=f&t=iA9OVLATcNUe0iod-0>

Ниже представлены макеты на рисунках 6 – 11



Рисунок 6 – страница Авторизации



Рисунок 7 – страница Регистрации



Рисунок 8 – страница Заказов

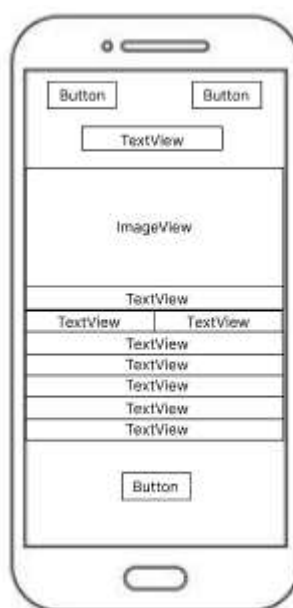


Рисунок 9 – страница Просмотра заказов

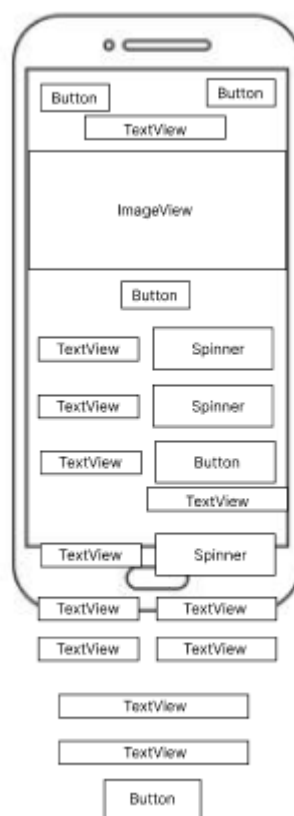


Рисунок 10 – страница Добавления/Редактирования заказов

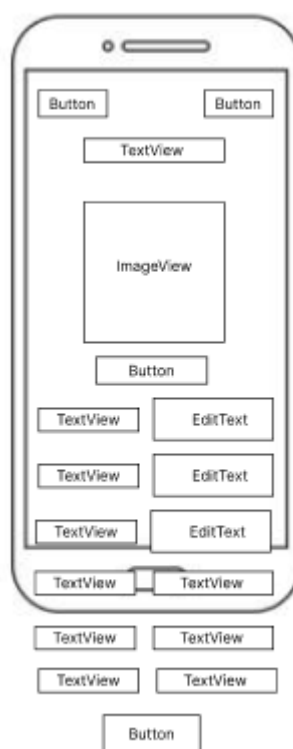


Рисунок 11 – страница Профиля

Навигационная схема на рисунке 12



Рисунок 12

4.7. Результаты работы программы

Регистрация нового пользователя

На рисунке 13 представлен экран регистрации

The screenshot shows the registration interface with the following elements:

- A red header bar with a "Назад" (Back) button in the bottom right corner.
- A title "Регистрация" (Registration) in bold black text.
- Four input fields for registration details:
 - Имя (Name): contains the text "всё"
 - Фамилия (Surname): contains the text "что"
 - Почта (Email): contains the text "смог"
 - Пароль (Password): contains the text "а" and a red asterisk icon indicating a password requirement.
- A red button labeled "Зарегистрироваться" (Register) at the bottom.

Рисунок 13

Вход в систему под новым пользователем представлен на рисунке 14



Авторизация

a

....

Войти

Регистрация

Рисунок 14

Основная страница с выводом заказов представлена на рисунке 15



Профиль Назад

Заказы

Поиск по заказам

Аппаратные камеры шкафы без оборудования

Готово

Петр Петров Петрович

20

Вы успешно авторизовались!

150 P

+

Рисунок 15

Фильтрация заказов представлена на рисунке 16



Рисунок 16

Просмотр заказа представлен на рисунке 17



Рисунок 17

Страница добавления и изменения заказов представлена на рисунке 18

[Профиль](#)[Назад](#)

Добавление/Редактирование



Заказчик:

Товар:

Дата заказа:

Выбрать дату

Дата не выбрана

Статус:

Цена:

500 Р

Рисунок 18

Профиль пользователя с изменениями представлен на рисунке 19

Профиль пользователя



Фамилия:

всё

Имя:

что

Отчество:

смог

Роль:

Пользователь

Логин:

a

Пароль:

aaaa

Сохранить изменения

Рисунок 19

4.8. Тестирование приложения

Для проверки корректности работы приложения были разработаны тест-кейсы, покрывающие ключевые сценарии использования. В таблице №7 представлены основные тест-кейсы с указанием предусловий, шагов выполнения и ожидаемых результатов.

Таблица №7

ID	Название тест-кейса	Предусловия	Шаги	Ожидаемый результат
ТС-01	Регистрация с коротким паролем	Открыт экран регистрации	1. Ввести имя, фамилию, отчество, логин 2. Ввести пароль "123" (3 символа) 3. Нажать кнопку "Зарегистрироваться"	Появляется сообщение: «Пароль должен содержать минимум 4 символа!», регистрация не выполнена
ТС-02	Авторизация с неверными данными	Открыт экран авторизации, в БД есть пользователь admin/123	1. Ввести логин "admin" 2. Ввести пароль "wrong" 3. Нажать кнопку "Войти"	Появляется сообщение: «Такого пользователя не найдено!», переход на главную не выполнен
ТС-03	Фильтрация заказов по статусу	Пользователь авторизован, в БД есть заказы со статусами "Готово" и "Не готово"	1. Открыть страницу заказов 2. В выпадающем списке «Статус» выбрать "Готово" 3. Наблюдать за списком	В списке отображаются только заказы со статусом "Готово", заказы со статусом "Не готово" скрыты
ТС-04	Поиск заказа по названию товара	Пользователь авторизован, в БД есть заказ с товаром "Аппаратные камеры"	1. Открыть страницу заказов 2. В поле поиска ввести "камеры" 3. Наблюдать за списком	В списке отображаются только заказы, содержащие в названии товара слово "камеры"
ТС-05	Создание нового заказа	Пользователь авторизован, в БД есть товары и статусы	1. Нажать кнопку "+" на странице заказов 2. Выбрать заказчика, товар, статус, дату 3. Нажать "Сохранить заказ"	Появляется сообщение «Заказ создан!», новый заказ отображается в списке на странице заказов

4.9. Выполнение тестов

По результатам выполнения тест-кейсов была составлена сводная таблица с указанием статуса прохождения каждого теста. Результаты тестирования представлены в таблице №8.

Таблица №8

ID	Название тест-кейса	Статус	Примечание
ТС-01	Регистрация с коротким паролем	Passed	Валидация длины пароля работает корректно, сообщение выводится
ТС-02	Авторизация с неверными данными	Passed	Проверка логина/пароля работает, доступ запрещён при ошибке
ТС-03	Фильтрация заказов по статусу	Passed	Spinner фильтрации работает, список обновляется мгновенно
ТС-04	Поиск заказа по названию товара	Passed	Поиск по мере ввода работает, регистронезависимый
ТС-05	Создание нового заказа	Passed	Заказ сохраняется в БД, отображается в списке с корректными данными

4.10. Инструкции по работе с программой

Как получить программу

- Приложение распространяется в виде установочного APK-файла (app-release.apk).
- Файл можно передать на устройство через USB-кабель, мессенджеры или облачное хранилище.

Требования к устройству

- Операционная система: Android 8.0 и выше
- Свободная память: не менее 100 МБ
- Разрешение экрана: от 720×1280 пикселей

Как установить

Скопируйте APK-файл на устройство Android.

В настройках устройства перейдите в раздел «Безопасность» или «Приложения».

Включите опцию «Установка из неизвестных источников» (для Android 8+ — разрешите установку для приложения «Проводник» или «Мои файлы»).

Откройте файловый менеджер, найдите скопированный APK-файл и нажмите на него.

Следуйте инструкциям установщика, нажмите «Установить», затем «Готово».

Как работать (по ролям)

Роль: Гость (неавторизованный пользователь)

Доступные действия:

- Просмотр экрана авторизации.
- Переход на экран регистрации.

Недоступно:

- Просмотр списка заказов.
- Создание и редактирование заказов.
- Доступ к профилю пользователя.

Примечание: Для работы с приложением необходимо зарегистрироваться или войти в систему.

Заключение

В ходе производственной практики разработано мобильное приложение для учёта заказов на Kotlin с использованием SQLite. Реализованы основные функции: авторизация, регистрация, создание и редактирование заказов, поиск и фильтрация, управление профилем. Все поставленные цели достигнуты, приложение соответствует требованиям предприятия.

В процессе работы закрепились навыки Android-разработки, работы с базами данных и тестирования. В перспективе возможна интеграция с сервером для синхронизации данных и добавление экспорта отчётов. Практика пройдена успешно, получен ценный опыт создания промышленного мобильного решения.

Список источников

- Android Developers [Электронный ресурс]. – URL: <https://developer.android.com> (дата обращения: 24.04.2026). – Текст : электронный.
- Room Persistence Library [Электронный ресурс]. – URL: <https://developer.android.com/training/data-storage/room> (дата обращения: 24.04.2026). – Текст : электронный.
- ГОСТ 19.701-90. ЕСПД. Схемы алгоритмов, программ, данных и систем [Электронный ресурс]. – Москва : Стандартинформ, 2010. – URL: <https://docs.cntd.ru/document/1200008963> (дата обращения: 24.04.2026).
- Glide [Электронный ресурс]. – URL: <https://bumptech.github.io/glide/> (дата обращения: 24.04.2026). – Текст : электронный.

В приложении весь листинг приложения на рисунках 1 – 75

```
package com.example.practic

import android.app.DatePickerDialog
import android.content.Intent
import android.net.Uri
import android.os.Bundle
import android.provider.MediaStore
import android.view.View
import android.widget.AdapterView
import android.widget.AdapterView.OnItemClickListener
import android.widget.ArrayAdapter
import android.widget.Button
import android.widget.ImageView
import android.widget.Spinner
import android.widget.TextView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import java.util.Calendar

class AddEditActivity : AppCompatActivity() {

    private lateinit var dbHelper: DBHelper
    private lateinit var imageView: ImageView
    private lateinit var spinnerUser: Spinner
    private lateinit var spinnerProduct: Spinner
    private lateinit var spinnerStatus: Spinner
    private lateinit var btnSelectDate: Button
    private lateinit var dateOrderTextView: TextView
    private lateinit var priceTextView: TextView
    private lateinit var productionTimeTextView: TextView
    private lateinit var descriptionTextView: TextView
    private lateinit var btnSave: Button
    private lateinit var btnAddImage: Button
```

Рисунок 1 – AddEditActivity.kt

```

private var selectedUserId: Int = 1
private var selectedProductId: Int = 1
private var selectedStatusId: Int = 1
private var selectedDate: String = getCurrentDate()
private var selectedImageUri: Uri? = null
private var orderId: Int = -1
private var currentUserId: Int = 1

private var usersList = mutableListOf<User>()
private var productsList = mutableListOf<Product>()
private var statusesList = mutableListOf<Status>()

companion object {
    const val REQUEST_IMAGE_PICK = 100
    const val EXTRA_ORDER_ID = "EXTRA_ORDER_ID"
}

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_add_edit)

    dbHelper = DBHelper(this, null)

    // Получаем ID текущего пользователя
    currentUserId = getCurrentUserId()

    orderId = intent.getIntExtra(EXTRA_ORDER_ID, -1)

    initView()
    setupClickListeners()
    loadDataFromDatabase()

    if (orderId > 0) {
        loadOrderData()
    }
}

private fun getCurrentUserId(): Int {
    val prefs = getSharedPreferences(MainActivity.PREFS_NAME,
MODE_PRIVATE)

```

Рисунок 2 – AddEditActivity.kt

```

        return prefs.getInt(MainActivity.KEY_USER_ID, 1)
    }

    private fun initViews() {
        imageView = findViewById(R.id.productImage_ImageView)
        spinnerUser = findViewById(R.id.spinnerUser)
        spinnerProduct = findViewById(R.id.spinnerProduct)
        spinnerStatus = findViewById(R.id.spinnerStatus)
        btnSelectDate = findViewById(R.id.btnSelectDate)
        dateOrderTextView = findViewById(R.id.dateOrder_TextView)
        priceTextView = findViewById(R.id.price_TextView)
        productionTimeTextView =
findViewById(R.id.productionTime_TextView)
        descriptionTextView =
findViewById(R.id.productDiscription_TextView)
        btnSave = findViewById(R.id.btnSave)
        btnAddImage = findViewById(R.id.btnAddImage)

        findViewById<Button>(R.id.btnBack).setOnClickListener { finish()
    }

        findViewById<Button>(R.id.profileButton).setOnClickListener {
            val intent = Intent(this, UserProfileActivity::class.java)
            intent.putExtra(UserProfileActivity.EXTRA_USER_ID,
currentUserId)
            startActivity(intent)
        }
    }

    private fun setupClickListeners() {
        btnSelectDate.setOnClickListener {
            showDatePicker()
        }

        btnAddImage.setOnClickListener {
            openImagePicker()
        }
    }

```

Рисунок 3 – AddEditActivity.kt

```

        spinnerProduct.onItemSelectedListener = object :
AdapterView.OnItemSelectedListener {
            override fun onItemSelected(parent: AdapterView<*>, view:
View?, position: Int, id: Long) {
                if (position < productsList.size) {
                    val product = productsList[position]
                    selectedProductId = product.Id_Product
                    updateProductInfo(product)
                }
            }
            override fun onNothingSelected(parent: AdapterView<*>) {}
        }

        spinnerUser.onItemSelectedListener = object :
AdapterView.OnItemSelectedListener {
            override fun onItemSelected(parent: AdapterView<*>, view:
View?, position: Int, id: Long) {
                if (position < usersList.size) {
                    selectedUserId = usersList[position].id
                }
            }
            override fun onNothingSelected(parent: AdapterView<*>) {}
        }

        spinnerStatus.onItemSelectedListener = object :
AdapterView.OnItemSelectedListener {
            override fun onItemSelected(parent: AdapterView<*>, view:
View?, position: Int, id: Long) {
                if (position < statusesList.size) {
                    selectedStatusId = statusesList[position].Id_Status
                }
            }
            override fun onNothingSelected(parent: AdapterView<*>) {}
        }

        btnSave.setOnClickListener {
            saveOrder()
        }
    }
}

```

Рисунок 4 – AddEditActivity.kt

```

        private fun loadDataFromDatabase() {
            userList = dbHelper.getAllUsers().toMutableList()
            if (userList.isEmpty()) {
                Toast.makeText(this, "Нет пользователей в БД!",
Toast.LENGTH_SHORT).show()
                return
            }

            productList = dbHelper.getAllProducts().toMutableList()
            if (productList.isEmpty()) {
                Toast.makeText(this, "Нет товаров в БД!",
Toast.LENGTH_SHORT).show()
                return
            }

            statusesList = dbHelper.getAllStatuses().toMutableList()
            if (statusesList.isEmpty()) {
                Toast.makeText(this, "Нет статусов в БД!",
Toast.LENGTH_SHORT).show()
                return
            }

            setupSpinners()
        }

        private fun setupSpinners() {
            val userNames = userList.map { "${it.firstname} ${it.lastname}"
}.toTypedArray()
            val userAdapter = ArrayAdapter(this,
android.R.layout.simple_spinner_item, userNames)

            userAdapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_
item)

            spinnerUser.adapter = userAdapter

            val productNames = productList.map { it.Name_Product
}.toTypedArray()
            val productAdapter = ArrayAdapter(this,
android.R.layout.simple_spinner_item, productNames)

```

Рисунок 5 – AddEditActivity.kt

```

        productAdapter.setDropDownViewResource(android.R.layout.simple_spinner_d
ropdown_item)

        spinnerProduct.adapter = productAdapter

        val statusNames = statusesList.map { it.Name_Status
}.toArray()

        val statusAdapter = ArrayAdapter(this,
android.R.layout.simple_spinner_item, statusNames)

statusAdapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdo
wn_item)

        spinnerStatus.adapter = statusAdapter
    }

    private fun updateProductInfo(product: Product) {
        priceTextView.text = "${product.Price_Product} ₴"
        productionTimeTextView.text = product.Production_Time
        descriptionTextView.text = product.Discription_Product

        val imageResId = resources.getIdentifier(
            product.Product_Photo, "drawable", packageName
        )
        if (imageResId != 0) {
            imageView.setImageResource(imageResId)
        }
    }

    private fun showDatePicker() {
        val calendar = Calendar.getInstance()
        val year = calendar.get(Calendar.YEAR)
        val month = calendar.get(Calendar.MONTH)
        val day = calendar.get(Calendar.DAY_OF_MONTH)

        DatePickerDialog(this, { _, selectedYear, selectedMonth,
selectedDay ->
            selectedDate = String.format("%04d-%02d-%02d", selectedYear,
selectedMonth + 1, selectedDay)
            dateOrderTextView.text = selectedDate
        }, year, month, day).show()
    }

```

Рисунок 6 – AddEditActivity.kt

```

        private fun openImagePicker() {
            val intent = Intent(Intent.ACTION_PICK,
MediaStore.Images.Media.EXTERNAL_CONTENT_URI)
            intent.type = "image/*"
            startActivityForResult(intent, REQUEST_IMAGE_PICK)
        }

        override fun onActivityResult(requestCode: Int, resultCode: Int,
data: Intent?) {
            super.onActivityResult(requestCode, resultCode, data)

            if (requestCode == REQUEST_IMAGE_PICK && resultCode == RESULT_OK
&& data != null) {
                selectedImageUri = data.data
                imageView.setImageURI(selectedImageUri)
            }
        }

        private fun saveOrder() {
            if (selectedDate.isEmpty()) {
                Toast.makeText(this, "Выберите дату заказа!",
Toast.LENGTH_SHORT).show()
                return
            }

            if (orderId > 0) {
                val updated = dbHelper.updateOrder(
                    orderId = orderId,
                    dateOrder = selectedDate,
                    idStatus = selectedStatusId,
                    idUser = selectedUserId,
                    idProduct = selectedProductId
                )

                if (updated) {
                    Toast.makeText(this, "Заказ обновлен!",
Toast.LENGTH_SHORT).show()
                    finish()
                }
            }
        }
    }
}

```

Рисунок 7 – AddEditActivity.kt

```

        else {
            Toast.makeText(this, "Ошибка при обновлении заказа",
Toast.LENGTH_SHORT).show()
        }
    } else {
        val newId = dbHelper.addOrder(
            dateOrder = selectedDate,
            idStatus = selectedStatusId,
            idUser = selectedUserId,
            idProduct = selectedProductId
        )

        if (newId > 0) {
            Toast.makeText(this, "Заказ создан!",
Toast.LENGTH_SHORT).show()
            finish()
        } else {
            Toast.makeText(this, "Ошибка при создании заказа",
Toast.LENGTH_SHORT).show()
        }
    }
}

private fun loadOrderData() {
    val order = dbHelper.getOrderById(orderId)
    if (order != null) {
        selectedDate = order.Date_Order
        dateOrderTextView.text = selectedDate

        val userIndex = usersList.indexOfFirst { it.id ==
order.Id_User }
        if (userIndex >= 0) {
            spinnerUser.setSelection(userIndex)
            selectedUserId = order.Id_User
        }

        val productIndex = productsList.indexOfFirst { it.Id_Product
== order.Id_Product }
        if (productIndex >= 0)

```

Рисунок 8 – AddEditActivity.kt

```

        {
            spinnerProduct.setSelection(productIndex)
            selectedProductId = order.Id_Product
            updateProductInfo(productsList[productIndex])
        }

        val statusIndex = statusesList.indexOfFirst { it.Id_Status
== order.Id_Status }
        if (statusIndex >= 0) {
            spinnerStatus.setSelection(statusIndex)
            selectedStatusId = order.Id_Status
        }
    }

    private fun getCurrentDate(): String {
        val calendar = Calendar.getInstance()
        return String.format("%04d-%02d-%02d",
            calendar.get(Calendar.YEAR),
            calendar.get(Calendar.MONTH) + 1,
            calendar.get(Calendar.DAY_OF_MONTH))
    }

    override fun onDestroy() {
        dbHelper.close()
        super.onDestroy()
    }
}

```

Рисунок 9 – AddEditActivity.kt

```

package com.example.practic

import android.content.ContentValues
import android.content.Context
import android.database.sqlite.SQLiteDatabase
import android.database.sqlite.SQLiteOpenHelper

class DBHelper(val context: Context, val factory:
SQLiteDatabase.CursorFactory?)
    : SQLiteOpenHelper(context, "BD1", factory, 3) {

    override fun onCreate(db: SQLiteDatabase?) {
        // Таблица Role
        db?.execSQL("""
            CREATE TABLE Role (
                Id_Role INTEGER PRIMARY KEY AUTOINCREMENT,
                Name_Role TEXT
            )
        """).trimIndent())

        // Таблица Status
        db?.execSQL("""
            CREATE TABLE Status (
                Id_Status INTEGER PRIMARY KEY AUTOINCREMENT,
                Name_Status TEXT
            )
        """).trimIndent())

        // Таблица Users
        db?.execSQL("""
            CREATE TABLE Users (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                firstname TEXT,
                lastname TEXT,
                middlename TEXT,
                login TEXT,
                password TEXT,
                user_Photo TEXT,
                Id_Role_User INTEGER,
                FOREIGN KEY(Id_Role_User) REFERENCES Role(Id_Role)
            )
        """).trimIndent())
    }
}

```

Рисунок 10 – DBHelper.kt

```

        """.trimIndent())

// Таблица Product
db?.execSQL("""
    CREATE TABLE Product (
        Id_Product INTEGER PRIMARY KEY AUTOINCREMENT,
        Name_Product TEXT,
        Discription_Product TEXT,
        Price_Product INTEGER,
        Production_Time TEXT,
        Product_Photo TEXT
    )
    """.trimIndent())

// Таблица Order
db?.execSQL("""
    CREATE TABLE "Order" (
        Id_Order INTEGER PRIMARY KEY AUTOINCREMENT,
        Date_Order TEXT,
        Id_Status INTEGER,
        Id_User INTEGER,
        Id_Product INTEGER,
        FOREIGN KEY(Id_Status) REFERENCES Status(Id_Status),
        FOREIGN KEY(Id_User) REFERENCES Users(id),
        FOREIGN KEY(Id_Product) REFERENCES Product(Id_Product)
    )
    """.trimIndent())

// Инициализация данных
initDefaultData(db)
}

private fun initDefaultData(db: SQLiteDatabase?) {
    // Статусы
    db?.insert("Status", null, ContentValues().apply {
        put("Name_Status", "Готово")
    })
    db?.insert("Status", null, ContentValues().apply {
        put("Name_Status", "Не готово")
    })
}

```

Рисунок 11 – DBHelper.kt

```

        // Роль
        db?.insert("Role", null, ContentValues().apply {
            put("Name_Role", "Пользователь")
        })

        // Товары
        val products = listOf(
            ProductData("Аппаратные камеры шкафы с оборудованием 1",
                "Аппаратные камеры и шкафы с оборудованием.",
                500, "15 дней", "hardware_cabinet"),
            ProductData("Аппаратные камеры шкафы без оборудования",
                "Аппаратные камеры и шкафы без оборудования",
                150, "2 дня", "photo2")
        )
        products.forEach { prod ->
            db?.insert("Product", null, ContentValues().apply {
                put("Name_Product", prod.name)
                put("Discription_Product", prod.description)
                put("Price_Product", prod.price)
                put("Production_Time", prod.productionTime)
                put("Product_Photo", prod.photoName)
            })
        }
    }

    override fun onUpgrade(db: SQLiteDatabase?, oldVersion: Int,
        newVersion: Int) {
        db?.execSQL("DROP TABLE IF EXISTS `Order`")
        db?.execSQL("DROP TABLE IF EXISTS Product")
        db?.execSQL("DROP TABLE IF EXISTS Users")
        db?.execSQL("DROP TABLE IF EXISTS Status")
        db?.execSQL("DROP TABLE IF EXISTS Role")
        onCreate(db)
    }

    // ♦ Добавление пользователя (регистрация)
    fun addUser(user: User): Long {
        val db = this.writableDatabase
        val values = ContentValues().apply {
            put("firstname", user.firstname)
            put("lastname", user.lastname)
        }
    }

```

Рисунок 12 – DBHelper.kt

```

        put("middlename", user.middlename)
        put("login", user.login)
        put("password", user.password)
        put("Id_Role_User", 1)
    }
    val id = db.insert("Users", null, values)
    db.close()
    return id
}

// Добавление заказа
fun addOrder(dateOrder: String, idStatus: Int, idUser: Int,
idProduct: Int): Long {
    val db = this.writableDatabase
    val values = ContentValues().apply {
        put("Date_Order", dateOrder)
        put("Id_Status", idStatus)
        put("Id_User", idUser)
        put("Id_Product", idProduct)
    }
    val id = db.insert("\"Order\"", null, values)
    db.close()
    return id
}

// Получение заказов с подробностями
fun getOrdersWithDetails(): List<OrderDisplay> {
    val result = mutableListOf<OrderDisplay>()
    val db = this.readableDatabase

    val query = """
SELECT
    o.Id_Order,
    u.firstname || ' ' || u.lastname || ' ' || u.middlename AS
full_name,
    o.Date_Order,
    s.Name_Status,
    p.Price_Product,
    p.Product_Photo,
    p.Name_Product,
    p.Discription_Product,

```

Рисунок 13 – DBHelper.kt

```

        p.Production_Time,
        s.Id_Status
    FROM "Order" o
    JOIN Users u ON o.Id_User = u.id
    JOIN Status s ON o.Id_Status = s.Id_Status
    JOIN Product p ON o.Id_Product = p.Id_Product
    ORDER BY o.Id_Order DESC
    """.trimIndent()

    val cursor = db.rawQuery(query, null)
    if (cursor.moveToFirst()) {
        do {
            // В цикле чтения курсора:
            result.add(OrderDisplay(
                Id_Order = cursor.getInt(0),
                userName = cursor.getString(1),
                dateOrder = cursor.getString(2),
                statusName = cursor.getString(3),
                statusId = cursor.getInt(9), // ♦ Добавлено:
читаем Id_Status

                price = cursor.getInt(4),
                productPhoto = cursor.getString(5),
                productName = cursor.getString(6),
                description = cursor.getString(7),
                productionTime = cursor.getString(8)
            ))
        } while (cursor.moveToNext())
    }
    cursor.close()
    db.close()
    return result
}

// Проверка логина
fun isLoginTaken(login: String): Boolean {
    val db = this.readableDatabase
    val cursor = db.rawQuery(
        "SELECT COUNT(*) FROM Users WHERE login = ?",
        arrayOf(login)
    )
    var isTaken = false

```

Рисунок 14 – DBHelper.kt

```

        if (cursor.moveToFirst()) {
            isTaken = cursor.getInt(0) > 0
        }
        cursor.close()
        db.close()
        return isTaken
    }

    //  Авторизация - ИСПРАВЛЕНО: добавлены user_Photo и Id_Role_User
    fun getUser(login: String, password: String): User? {
        val db = this.readableDatabase
        val cursor = db.rawQuery(
            "SELECT * FROM Users WHERE login = ? AND password = ?",
            arrayOf(login, password)
        )

        var user: User? = null
        if (cursor.moveToFirst()) {
            user = User(
                id = cursor.getInt(cursor.getColumnIndexOrThrow("id")),
                firstname =
                    cursor.getString(cursor.getColumnIndexOrThrow("firstname")),
                lastname =
                    cursor.getString(cursor.getColumnIndexOrThrow("lastname")),
                middlename =
                    cursor.getString(cursor.getColumnIndexOrThrow("middlename")),
                login =
                    cursor.getString(cursor.getColumnIndexOrThrow("login")),
                password =
                    cursor.getString(cursor.getColumnIndexOrThrow("password")),
                user_Photo =
                    cursor.getString(cursor.getColumnIndexOrThrow("user_Photo")),
                Id_Role_User =
                    cursor.getInt(cursor.getColumnIndexOrThrow("Id_Role_User"))
            )
        }
        cursor.close()
        db.close()
        return user
    }

```

Рисунок 15 – DBHelper.kt

```

//  Получить всех пользователей - ИСПРАВЛЕНО: добавлены
user_Photo и Id_Role_User

fun getAllUsers(): List<User> {
    val users = mutableListOf<User>()
    val db = this.readableDatabase
    val cursor = db.rawQuery("SELECT * FROM Users", null)

    if (cursor.moveToFirst()) {
        do {
            users.add(User(
                id =
cursor.getInt(cursor.getColumnIndexOrThrow("id")),
                firstname =
cursor.getString(cursor.getColumnIndexOrThrow("firstname")),
                lastname =
cursor.getString(cursor.getColumnIndexOrThrow("lastname")),
                middlename =
cursor.getString(cursor.getColumnIndexOrThrow("middlename")),
                login =
cursor.getString(cursor.getColumnIndexOrThrow("login")),
                password =
cursor.getString(cursor.getColumnIndexOrThrow("password")),
                user_Photo =
cursor.getString(cursor.getColumnIndexOrThrow("user_Photo")),
                Id_Role_User =
cursor.getInt(cursor.getColumnIndexOrThrow("Id_Role_User"))
            ))
        } while (cursor.moveToNext())
    }
    cursor.close()
    db.close()
    return users
}

// Получить все товары
fun getAllProducts(): List<Product> {
    val products = mutableListOf<Product>()
    val db = this.readableDatabase
    val cursor = db.rawQuery("SELECT * FROM Product", null)

```

Рисунок 16 – DBHelper.kt

```

        if (cursor.moveToFirst()) {
            do {
                products.add(Product(
                    Id_Product =
cursor.getInt(cursor.getColumnIndexOrThrow("Id_Product")),
                    Name_Product =
cursor.getString(cursor.getColumnIndexOrThrow("Name_Product")),
                    Discription_Product =
cursor.getString(cursor.getColumnIndexOrThrow("Discription_Product")),
                    Price_Product =
cursor.getInt(cursor.getColumnIndexOrThrow("Price_Product")),
                    Production_Time =
cursor.getString(cursor.getColumnIndexOrThrow("Production_Time")),
                    Product_Photo =
cursor.getString(cursor.getColumnIndexOrThrow("Product_Photo"))
                ))
            } while (cursor.moveToNext())
        }
        cursor.close()
        db.close()
        return products
    }

    // Получить все статусы
    fun getAllStatuses(): List<Status> {
        val statuses = mutableListOf<Status>()
        val db = this.readableDatabase
        val cursor = db.rawQuery("SELECT * FROM Status", null)

        if (cursor.moveToFirst()) {
            do {
                statuses.add(Status(
                    Id_Status =
cursor.getInt(cursor.getColumnIndexOrThrow("Id_Status")),
                    Name_Status =
cursor.getString(cursor.getColumnIndexOrThrow("Name_Status"))
                ))
            } while (cursor.moveToNext())
        }
        cursor.close()
        db.close()
    }

```

Рисунок 17 – DBHelper.kt

```

        return statuses
    }
    // Получить заказ по ID
    fun getOrderById(orderId: Int): Order? {
        val db = this.readableDatabase
        val cursor = db.rawQuery("SELECT * FROM \"Order\" WHERE Id_Order = ?", arrayOf(orderId.toString()))

        var order: Order? = null
        if (cursor.moveToFirst()) {
            order = Order(
                Id_Order =
                cursor.getInt(cursor.getColumnIndexOrThrow("Id_Order")),
                Date_Order =
                cursor.getString(cursor.getColumnIndexOrThrow("Date_Order")),
                Id_Status =
                cursor.getInt(cursor.getColumnIndexOrThrow("Id_Status")),
                Id_User =
                cursor.getInt(cursor.getColumnIndexOrThrow("Id_User")),
                Id_Product =
                cursor.getInt(cursor.getColumnIndexOrThrow("Id_Product"))
            )
        }
        cursor.close()
        db.close()
        return order
    }

    // Обновить заказ
    fun updateOrder(
        orderId: Int,
        dateOrder: String,
        idStatus: Int,
        idUser: Int,
        idProduct: Int
    ): Boolean {
        val db = this.writableDatabase
        val values = ContentValues().apply {
            put("Date_Order", dateOrder)
            put("Id_Status", idStatus)
            put("Id_User", idUser)

```

Рисунок 18 – DBHelper.kt

```

        put("Id_Product", idProduct)
    }
    val rowsAffected = db.update("\Order\\"", values, "Id_Order = ?", arrayOf(orderId.toString()))
    db.close()
    return rowsAffected > 0
}

// Получить пользователя по ID
fun getUserById(userId: Int): User? {
    val db = this.readableDatabase
    val cursor = db.rawQuery(
        "SELECT * FROM Users WHERE id = ?",
        arrayOf(userId.toString())
    )

    var user: User? = null
    if (cursor.moveToFirst()) {
        user = User(
            id = cursor.getInt(cursor.getColumnIndexOrThrow("id")),
            firstname =
cursor.getString(cursor.getColumnIndexOrThrow("firstname")),
            lastname =
cursor.getString(cursor.getColumnIndexOrThrow("lastname")),
            middlename =
cursor.getString(cursor.getColumnIndexOrThrow("middlename")),
            login =
cursor.getString(cursor.getColumnIndexOrThrow("login")),
            password =
cursor.getString(cursor.getColumnIndexOrThrow("password")),
            user_Photo =
cursor.getString(cursor.getColumnIndexOrThrow("user_Photo")),
            Id_Role_User =
cursor.getInt(cursor.getColumnIndexOrThrow("Id_Role_User"))
        )
    }
    cursor.close()
    db.close()
    return user
}

```

Рисунок 19 – DBHelper.kt

```

// Получить роль по ID
fun getRoleById(roleId: Int): Role? {
    val db = this.readableDatabase
    val cursor = db.rawQuery(
        "SELECT * FROM Role WHERE Id_Role = ?",
        arrayOf(roleId.toString())
    )
    var role: Role? = null
    if (cursor.moveToFirst()) {
        role = Role(
            Id_Role =
cursor.getInt(cursor.getColumnIndexOrThrow("Id_Role")),
            Name_Role =
cursor.getString(cursor.getColumnIndexOrThrow("Name_Role"))
        )
    }
    cursor.close()
    db.close()
    return role
}

// Обновить пользователя
fun updateUser(
    userId: Int,
    firstname: String,
    lastname: String,
    middlename: String,
    userPhoto: String?
): Boolean {
    val db = this.writableDatabase
    val values = ContentValues().apply {
        put("firstname", firstname)
        put("lastname", lastname)
        put("middlename", middlename)
        // Сохраняем путь к фото (или null если не изменено)
        put("user_Photo", userPhoto)
    }
    val rowsAffected = db.update("Users", values, "id = ?",
arrayOf(userId.toString()))
    db.close()
    return rowsAffected > 0
}

```

Рисунок 20 – DBHelper.kt

```

        private data class ProductData(
            val name: String,
            val description: String,
            val price: Int,
            val productionTime: String,
            val photoName: String
        )
    }

```

Рисунок 21 – DBHelper.kt

```

package com.example.practic

import android.content.Intent
import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.TextView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

    companion object {
        const val PREFS_NAME = "app_prefs"
        const val KEY_USER_ID = "current_user_id"
        const val KEY_LOGIN = "current_login"
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val linkToReg: TextView = findViewById(R.id.toReg)
        val userLogin: EditText = findViewById(R.id.user_login)
        val userPassword: EditText = findViewById(R.id.user_password)
        val button: Button = findViewById(R.id.button_auth)

        // Переход на регистрацию
        linkToReg.setOnClickListener {
            startActivity(Intent(this, RegActivity::class.java))
        }
    }
}

```

Рисунок 22 – MainActivity.kt

```

        button.setOnClickListener {
            val login = userLogin.text.toString().trim()
            val password = userPassword.text.toString().trim()
            if (login.isEmpty() || password.isEmpty()) {
                Toast.makeText(this, "Не все поля заполнены!",
Toast.LENGTH_LONG).show()
                return@setOnClickListener
            }
            val db = DBHelper(this, null)
            val user = db.getUser(login, password)
            if (user != null) {
                Toast.makeText(this, "Вы успешно авторизовались!",
Toast.LENGTH_LONG).show()
                val prefs = getSharedPreferences(PREFS_NAME,
MODE_PRIVATE)

                prefs.edit()
                    .putInt(KEY_USER_ID, user.id)
                    .putString(KEY_LOGIN, user.login)
                    .apply()
                userLogin.text.clear()
                userPassword.text.clear()
                // Переход на страницу заказов
                startActivity(Intent(this, ProductActivity::class.java))
                finish()
            } else {
                Toast.makeText(this, "Такого пользователя не найдено!",
Toast.LENGTH_LONG).show()
                debugUsers(db)
            }
            db.close()
        }
    }

    private fun debugUsers(db: DBHelper) {
        val allUsers = db.getAllUsers()
        android.util.Log.d("AUTH_DEBUG", "Пользователей в БД:
${allUsers.size}")
        allUsers.forEach { u ->
            android.util.Log.d("AUTH_DEBUG", "→ id=${u.id},
login='${u.login}'")
        }
    }
}

```

Рисунок 23 – MainActivity.kt

```
package com.example.practic

class Order(val Id_Order: Int,
            val Date_Order: String,
            val Id_Status: Int,
            val Id_User: Int,
            val Id_Product: Int) {

}
```

Рисунок 24 – Class Order

```
package com.example.practic

class Product(val Id_Product: Int,
              val Name_Product: String,
              val Discription_Product: String,
              val Price_Product: Int,
              val Production_Time: String,
              val Product_Photo: String) {

}
```

Рисунок 25 – Class Product

```
package com.example.practic

data class OrderDisplay(
    val Id_Order: Int,
    val userName: String,
    val dateOrder: String,
    val statusName: String,
    val statusId: Int,
    val price: Int,
    val productPhoto: String,
    val productName: String,
    val description: String,
    val productionTime: String
)
```

Рисунок 26 – Class OrderDisplay

```
package com.example.practic

class Role(val Id_Role: Int,
           val Name_Role: String ) {

}
```

Рисунок 27 – Class Role

```
package com.example.practic

class Status(val Id_Status: Int,
             val Name_Status: String ) {

}
```

Рисунок 28 – Class Status

```
package com.example.practic

data class User(
    val id: Int,
    val firstname: String,
    val lastname: String,
    val middlename: String,
    val login: String,
    val password: String,
    val user_Photo: String? = null,
    val Id_Role_User: Int
)
```

Рисунок 29 – Class User

```
package com.example.practic

import android.content.Intent
import android.os.Bundle
import android.text.Editable
import android.text.TextWatcher
import android.widget.*
import androidx.appcompat.app.AppCompatActivity
import androidx.recyclerview.widget.LinearLayoutManager
import androidx.recyclerview.widget.RecyclerView

class ProductActivity : AppCompatActivity() {

    private lateinit var dbHelper: DBHelper
    private lateinit var adapter: ProductAdapter
    private val ordersList = mutableListOf<OrderDisplay>()
    private val filteredOrdersList = mutableListOf<OrderDisplay>()
    private var currentUserId: Int = 1

}
```

Рисунок 30 – ProductActivity.kt

```

        // Элементы управления поиском
        private lateinit var searchEditText: EditText
        private lateinit var statusFilterSpinner: Spinner
        private lateinit var btnClearFilter: Button
        // Данные для фильтра
        private var statusesList = mutableListOf<Status>()
        private var selectedStatusId: Int = 0 // 0 = все статусы
        override fun onCreate(savedInstanceState: Bundle?) {
            super.onCreate(savedInstanceState)
            setContentView(R.layout.activity_product)
            currentUserId = getCurrentUserId()
            dbHelper = DBHelper(this, null)
            initView()
            initBaseData()
            initTestOrders()
            loadStatusesForFilter()
            setupRecyclerView()
            setupSearchAndFilter()
            loadOrders()
            // Кнопки навигации
            findViewById<Button>(R.id.btnBack)?.setOnClickListener {
finish() }

            findViewById<Button>(R.id.profileButton)?.setOnClickListener {
                val intent = Intent(this, UserProfileActivity::class.java)
                intent.putExtra(UserProfileActivity.EXTRA_USER_ID,
currentUserId)

                startActivity(intent)
            }
            findViewById<Button>(R.id.AddButton)?.setOnClickListener {
                val intent = Intent(this, AddEditActivity::class.java)
                intent.putExtra("CURRENT_USER_ID", currentUserId)
                startActivity(intent)
            }
        }
        private fun getCurrentUserId(): Int {
            val prefs = getSharedPreferences(MainActivity.PREFS_NAME,
MODE_PRIVATE)

            return prefs.getInt(MainActivity.KEY_USER_ID, 1)
        }
    }

```

Рисунок 31 – ProductActivity.kt

```

private fun initView() {
    searchText = findViewById(R.id.searchEditText)
    statusFilterSpinner = findViewById(R.id.statusFilterSpinner)
    btnClearFilter = findViewById(R.id.btnClearFilter)
}

// ♦ Инициализация справочных данных
private fun initBaseData() {
    val db = dbHelper.writableDatabase
    // Status
    var c = db.rawQuery("SELECT COUNT(*) FROM Status", null)
    c.moveToFirst()
    if (c.getInt(0) == 0) {
        db.insert("Status", null,
android.content.ContentValues().apply {
            put("Name_Status", "Готово")
        })
        db.insert("Status", null,
android.content.ContentValues().apply {
            put("Name_Status", "Не готово")
        })
    }
    c.close()
    // Role
    c = db.rawQuery("SELECT COUNT(*) FROM Role", null)
    c.moveToFirst()
    if (c.getInt(0) == 0) {
        db.insert("Role", null,
android.content.ContentValues().apply {
            put("Name_Role", "Пользователь")
        })
    }
    c.close()
    // User (admin)
    c = db.rawQuery("SELECT COUNT(*) FROM Users WHERE id=1", null)
    c.moveToFirst()
    if (c.getInt(0) == 0) {
        db.insert("Users", null,
android.content.ContentValues().apply {
            put("id", 1)
            put("firstname", "Иван")
            put("lastname", "Иванов")
        })
    }
}

```

Рисунок 32 – ProductActivity.kt

```

        put("middlename", "Иванович")
        put("login", "admin")
        put("password", "123")
        put("Id_Role_User", 1)
    })
}
c.close()
// Products
c = db.rawQuery("SELECT COUNT(*) FROM Product", null)
c.moveToFirst()
if (c.getInt(0) == 0) {
    listOf(
        "Товар 1" to 500,
        "Товар 2" to 150,
        "Товар 3" to 999
    ).forEachIndexed { i, (name, price) ->
        db.insert("Product", null,
android.content.ContentValues().apply {
            put("Name_Product", name)
            put("Discription_Product", "Описание $name")
            put("Price_Product", price)
            put("Production_Time", "${i+1} дней")
            put("Product_Photo", if (i == 0) "hardware_cabinet"
else "photo2")
        })
    }
}
c.close()
db.close()
}

// ♦ Создание тестовых заказов
private fun initTestOrders() {
    val db = dbHelper.writableDatabase
    var c = db.rawQuery("SELECT COUNT(*) FROM \"Order\"", null)
    c.moveToFirst()
    if (c.getInt(0) == 0) {
        db.insert("\"Order\"", null,
android.content.ContentValues().apply {
            put("Date_Order", "2026-04-20 10:30")
            put("Id_Status", 1); put("Id_User", 1);
put("Id_Product", 1)

```

Рисунок 33 – ProductActivity.kt

```

        })
        db.insert("\"Order\"", null,
android.content.ContentValues().apply {
            put("Date_Order", "2026-04-21 09:15")
            put("Id_Status", 2); put("Id_User", 1);
put("Id_Product", 2)
        })
        db.insert("\"Order\"", null,
android.content.ContentValues().apply {
            put("Date_Order", "2026-04-21 14:00")
            put("Id_Status", 2); put("Id_User", 1);
put("Id_Product", 3)
        })
    }
    c.close()
    db.close()
}

// ♦ Загрузка статусов для фильтра
private fun loadStatusesForFilter() {
    statusesList = dbHelper.getAllStatuses().toMutableList()
    // Добавляем опцию "Все статусы"
    val allStatuses = mutableListOf(Status(0, "Все статусы"))
    allStatuses.addAll(statusesList)
    // Настраиваем Spinner
    val statusNames = allStatuses.map { it.Name_Status
}.toTypedArray()
    val adapter = ArrayAdapter(this,
android.R.layout.simple_spinner_item, statusNames)

    adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item
)

    statusFilterSpinner.adapter = adapter
    // Обработчик выбора статуса
    statusFilterSpinner.onItemSelectedListener = object :
AdapterView.OnItemSelectedListener {
        override fun onItemSelected(parent: AdapterView<*>, view:
android.view.View?, position: Int, id: Long) {
            selectedStatusId = allStatuses[position].Id_Status
            applyFilters()
        }
        override fun onNothingSelected(parent: AdapterView<*>) {}
    }
}

```

Рисунок 34 – ProductActivity.kt

```

    }

}

// ♦ Настройка поиска и фильтрации
private fun setupSearchAndFilter() {
    // Поиск по мере ввода (с небольшой задержкой)
    searchEditText.addTextChangedListener(object : TextWatcher {
        override fun afterTextChanged(s: Editable?) {
            applyFilters()
        }
        override fun beforeTextChanged(s: CharSequence?, start: Int,
count: Int, after: Int) {}
        override fun onTextChanged(s: CharSequence?, start: Int,
before: Int, count: Int) {}
    })
    // Кнопка очистки фильтров
    btnClearFilter.setOnClickListener {
        searchEditText.text.clear()
        statusFilterSpinner.setSelection(0) // "Все статусы"
        applyFilters()
    }
}

// ♦ Применение фильтров к списку
private fun applyFilters() {
    val searchQuery =
searchEditText.text.toString().trim().lowercase()
    filteredOrdersList.clear()
    for (order in ordersList) {
        // Фильтр по названию товара
        val matchesSearch = searchQuery.isEmpty() ||
            order.productName.lowercase().contains(searchQuery)
        // Фильтр по статусу (0 = все)
        val matchesStatus = selectedStatusId == 0 ||
            order.statusId == selectedStatusId
        if (matchesSearch && matchesStatus) {
            filteredOrdersList.add(order)
        }
    }
    adapter.updateList(filteredOrdersList)
    // Показать сообщение если ничего не найдено

```

Рисунок 35 – ProductActivity.kt

```

        if (filteredOrdersList.isEmpty() && (searchQuery.isNotEmpty() ||
selectedStatusId != 0)) {
            Toast.makeText(this, "Ничего не найдено",
Toast.LENGTH_SHORT).show()
        }
    }

    // ♦ Настройка RecyclerView
    private fun setupRecyclerView() {
        val rv = findViewById<RecyclerView>(R.id.Product_List)
        rv.layoutManager = LinearLayoutManager(this)
        adapter = ProductAdapter(filteredOrdersList, this)
        rv.adapter = adapter
    }

    // ♦ Загрузка заказов из БД
    private fun loadOrders() {
        ordersList.clear()
        val loaded = dbHelper.getOrdersWithDetails()
        ordersList.addAll(loaded)
        // Применяем текущие фильтры
        applyFilters()
    }

    override fun onResume() {
        super.onResume()
        loadOrders() // Обновляем при возврате в активность
    }

    override fun onDestroy() {
        dbHelper.close()
        super.onDestroy()
    }
}

```

Рисунок 36 – ProductActivity.kt

```

package com.example.practic

import android.content.Context
import android.content.Intent
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.Button
import android.widget.ImageView
import android.widget.TextView
import androidx.recyclerview.widget.RecyclerView

class ProductAdapter(
    private var orders: List<OrderDisplay>,
    private val context: Context
) : RecyclerView.Adapter<ProductAdapter.MyViewHolder>() {

    class MyViewHolder(view: View) : RecyclerView.ViewHolder(view) {
        val image: ImageView =
view.findViewById(R.id.product_list_image)
        val title: TextView = view.findViewById(R.id.product_list_name)
        val status: TextView =
view.findViewById(R.id.product_list_status)
        val price: TextView = view.findViewById(R.id.product_list_price)
        val userInfo: TextView =
view.findViewById(R.id.product_list_user)
        val dateInfo: TextView =
view.findViewById(R.id.product_list_date)
        val btn: Button = view.findViewById(R.id.product_list_button)
    }

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
MyViewHolder {
        val view = LayoutInflater.from(parent.context)
            .inflate(R.layout.product_in_list, parent, false)
        return MyViewHolder(view)
    }

    override fun onBindViewHolder(holder: MyViewHolder, position: Int) {
        val order = orders[position]
    }
}

```

Рисунок 37 – ProductAdapter.kt

```

        holder.title.text = order.productName
        holder.price.text = "${order.price} ₴"
        holder.status.text = order.statusName
        holder.userInfo.text = order.userName
        holder.dateInfo.text = order.dateOrder
        // Цвет статьи
        val statusColor = if (order.statusName == "Готово")
            android.R.color.holo_green_dark
        else
            android.R.color.holo_red_dark

        holder.status.setTextColor(androidx.core.content.ContextCompat.getColor(context, statusColor))

        // Загрузка изображения
        val imageResId = context.resources.getIdentifier(
            order.productPhoto, "drawable", context.packageName
        )
        if (imageResId != 0) {
            holder.image.setImageResource(imageResId)
        } else {

holder.image.setImageResource(android.R.drawable.ic_menu_gallery)
        }
        // Переход на детальный просмотр
        holder.btn.setOnClickListener {
            val intent = Intent(context,
WatchActivity::class.java).apply {
                putExtra("Order_Id", order.Id_Order)
                putExtra("Product_Photo", order.productPhoto)
                putExtra("ProductName", order.productName)
                putExtra("UserName", order.userName)
                putExtra("Date_Order", order.dateOrder)
                putExtra("Production_Time", order.productionTime)
                putExtra("Name_Status", order.statusName)
                putExtra("Discription_Product", order.description)
                putExtra("Price_Product", order.price)
            }
            context.startActivity(intent)
        }
    }
}

```

Рисунок 38 – ProductAdapter.kt

```

        override fun getItemCount(): Int = orders.size
        // ♦ Метод для обновления списка с фильтрацией
        fun updateList(newOrders: List<OrderDisplay>) {
            orders = newOrders
            notifyDataSetChanged()
        }
    }
}

```

Рисунок 39 – ProductAdapter.kt

```

package com.example.practic

import android.content.Intent
import android.graphics.Bitmap
import android.graphics.BitmapFactory
import android.net.Uri
import android.os.Bundle
import android.provider.MediaStore
import android.widget.Button
import android.widget.EditText
import android.widget.ImageView
import android.widget.TextView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import java.io.File
import java.io.FileOutputStream
import java.io.IOException

class UserProfileActivity : AppCompatActivity() {

    private lateinit var dbHelper: DBHelper
    private lateinit var userPhotoImageView: ImageView
    private lateinit var userLastnameEditText: EditText
    private lateinit var userFirstnameEditText: EditText
    private lateinit var userMiddlenameEditText: EditText
    private lateinit var userRoleTextView: TextView
    private lateinit var userLoginTextView: TextView
    private lateinit var userPasswordTextView: TextView
    private lateinit var btnSave: Button
    private lateinit var btnAddImage: Button

```

Рисунок 40 – UserProfilActivity.kt

```

private var currentUser: User? = null
private var selectedImageUri: Uri? = null
private var currentUserId: Int = -1
companion object {
    const val REQUEST_IMAGE_PICK = 100
    const val EXTRA_USER_ID = "EXTRA_USER_ID"
}
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_user_profile)
    dbHelper = DBHelper(this, null)
    currentUserId = intent.getIntExtra(EXTRA_USER_ID,
getCurrentUserId())
    initView()
    setupClickListeners()
    loadUserData()
}
private fun getCurrentUserId(): Int {
    val prefs = getSharedPreferences(MainActivity.PREFS_NAME,
MODE_PRIVATE)
    return prefs.getInt(MainActivity.KEY_USER_ID, 1)
}
private fun initView() {
    userPhotoImageView = findViewById(R.id.userPhoto_ImageView)
    userLastnameEditText = findViewById(R.id.userLastname_EditText)
    userFirstnameEditText =
findViewById(R.id.userFirstname_EditText)
    userMiddlenameEditText =
findViewById(R.id.userMiddlename_EditText)
    userRoleTextView = findViewById(R.id.userRole_TextView)
    userLoginTextView = findViewById(R.id.userLogin_TextView)
    userPasswordTextView = findViewById(R.id.userPassword_TextView)
    btnSave = findViewById(R.id.btnSave)
    btnAddImage = findViewById(R.id.btnAddImage)
    findViewById<Button>(R.id.btnBack).setOnClickListener { finish()
}

    findViewById<Button>(R.id.profileButton).setOnClickListener {
        // Можно оставить пустым
    }
}

```

Рисунок 41 – UserProfileActivity.kt

```

        private fun setupClickListeners() {
            btnAddImage.setOnClickListener { openImagePicker() }
            btnSave.setOnClickListener { saveChanges() }
        }
        private fun loadUserData() {
            currentUser = dbHelper.getUserById(currentUserId)
            if (currentUser != null) {
                userFirstnameEditText.setText(currentUser!!.firstname)
                userLastnameEditText.setText(currentUser!!.lastname)
                userMiddlenameEditText.setText(currentUser!!.middlename)
                userLoginTextView.text = currentUser!!.login
                userPasswordTextView.text = currentUser!!.password
                val role = dbHelper.getRoleById(currentUser!!.Id_Role_User)
                userRoleTextView.text = role?.Name_Role ?: "Неизвестно"
                // Загружаем фото пользователя
                loadUserPhoto(currentUser!!.user_Photo)
            } else {
                Toast.makeText(this, "Пользователь не найден",
                    Toast.LENGTH_SHORT).show()
                finish()
            }
        }
        private fun loadUserPhoto(photoPath: String?) {
            if (!photoPath.isNullOrEmpty()) {
                val photoFile = File(photoPath)
                if (photoFile.exists()) {
                    val bitmap = BitmapFactory.decodeFile(photoPath)
                    userPhotoImageView.setImageBitmap(bitmap)
                } else {
                    // Если файл не найден, используем заглушку
                    userPhotoImageView.setImageResource(android.R.drawable.ic_menu_gallery)
                }
            } else {
                userPhotoImageView.setImageResource(android.R.drawable.ic_menu_gallery)
            }
        }
    }

```

Рисунок 42 – UserProfileActivity.kt

```

        private fun openImagePicker() {
            val intent = Intent(Intent.ACTION_PICK,
MediaStore.Images.Media.EXTERNAL_CONTENT_URI)
            intent.type = "image/*"
            startActivityForResult(intent, REQUEST_IMAGE_PICK)
        }

        override fun onActivityResult(requestCode: Int, resultCode: Int,
data: Intent?) {
            super.onActivityResult(requestCode, resultCode, data)
            if (requestCode == REQUEST_IMAGE_PICK && resultCode == RESULT_OK
&& data != null) {
                selectedImageUri = data.data
                try {
                    val bitmap =
MediaStore.Images.Media.getBitmap(contentResolver, selectedImageUri)
                    userPhotoImageView.setImageBitmap(bitmap)
                } catch (e: IOException) {
                    e.printStackTrace()
                    Toast.makeText(this, "Ошибка загрузки изображения",
Toast.LENGTH_SHORT).show()
                }
            }
        }

        private fun saveChanges() {
            if (currentUser == null) return
            val firstname = userFirstnameEditText.text.toString().trim()
            val lastname = userLastnameEditText.text.toString().trim()
            val middlename = userMiddlenameEditText.text.toString().trim()
            if (firstname.isEmpty() || lastname.isEmpty()) {
                Toast.makeText(this, "Имя и фамилия обязательны!",
Toast.LENGTH_SHORT).show()
                return
            }
            var photoPath = currentUser!!.user_Photo
            // Если выбрано новое фото, сохраняем его
            if (selectedImageUri != null) {
                photoPath = saveImageToInternalStorage(selectedImageUri!!)
                if (photoPath == null) {
                    Toast.makeText(this, "Ошибка сохранения фото",
Toast.LENGTH_SHORT).show()

```

Рисунок 43 – UserProfileActivity.kt

```

        return
    }
}

val updated = dbHelper.updateUser(
    userId = currentUserId,
    firstname = firstname,
    lastname = lastname,
    middlename = middlename,
    userPhoto = photoPath
)

if (updated) {
    Toast.makeText(this, "Изменения сохранены!",
Toast.LENGTH_SHORT).show()
    finish()
} else {
    Toast.makeText(this, "Ошибка при сохранении",
Toast.LENGTH_SHORT).show()
}
}

private fun saveImageToInternalStorage(imageUri: Uri): String? {
    return try {
        val bitmap =
MediaStore.Images.Media.getBitmap(contentResolver, imageUri)
        val filename = "user_photo_${currentUserId}.jpg"
        val file = File(filesDir, filename)
        FileOutputStream(file).use { fos ->
            bitmap.compress(Bitmap.CompressFormat.JPEG, 90, fos)
        }
        file.absolutePath
    } catch (e: Exception) {
        e.printStackTrace()
        null
    }
}

override fun onDestroy() {
    dbHelper.close()
    super.onDestroy()
}
}

```

Рисунок 44 – UserProfileActivity.kt

```

package com.example.practic

import android.content.Intent
import android.os.Bundle
import android.widget.Button
import android.widget.ImageView
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity
import androidx.core.content.ContextCompat

class WatchActivity : AppCompatActivity() {
    private lateinit var dbHelper: DBHelper
    private var currentUserId: Int = 1
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_watch)
        dbHelper = DBHelper(this, null)
        currentUserId = getCurrentUserId()
        // Кнопки
        findViewById<Button>(R.id.btnBack).setOnClickListener { finish()
}

        findViewById<Button>(R.id.profileButton).setOnClickListener {
            val intent = Intent(this, UserProfileActivity::class.java)
            intent.putExtra(UserProfileActivity.EXTRA_USER_ID,
currentUserId)

            startActivity(intent)
        }
        findViewById<Button>(R.id.red_button).setOnClickListener {
            val intent = Intent(this, AddEditActivity::class.java)
            intent.putExtra("CURRENT_USER_ID", currentUserId)
            startActivity(intent)
        }
        // UI элементы
        val productImage: ImageView =
findViewById(R.id.productImage_ImageView)
        val productName: TextView =
findViewById(R.id.productName_TextView)
        val userName: TextView = findViewById(R.id.userName_TextView)
        val dateOrder: TextView = findViewById(R.id.dateOrder_TextView)
        val productionTime: TextView =
findViewById(R.id.productionTime_TextView)

```

Рисунок 45 – WatchActivity.kt

```

        val productStatus: TextView =
findViewById(R.id.productStatus_TextView)
        val productDescription: TextView =
findViewById(R.id.productDiscription_TextView)
        val priceProduct: TextView = findViewById(R.id.price_TextView)
        // Данные из Intent
        val photoName = intent.getStringExtra("Product_Photo")
        productName.text = intent.getStringExtra("ProductName")
        userName.text = intent.getStringExtra("UserName")
        dateOrder.text = intent.getStringExtra("Date_Order")
        productionTime.text = intent.getStringExtra("Production_Time")
        productStatus.text = intent.getStringExtra("Name_Status")
        productDescription.text =
intent.getStringExtra("Discription_Product")
        priceProduct.text = "${intent.getIntExtra("Price_Product", 0)}
Р"

        // Загрузка изображения
        val imageResId = resources.getIdentifier(photoName, "drawable",
packageName)
        if (imageResId != 0) {
            productImage.setImageResource(imageResId)
        }
        // Цвет статуса
        val status = intent.getStringExtra("Name_Status")
        val colorRes = if (status == "Готово")
            android.R.color.holo_green_dark
        else
            android.R.color.holo_red_dark
        productStatus.setTextColor(ContextCompat.getColor(this,
colorRes))
    }
    private fun getCurrentUserId(): Int {
        val prefs = getSharedPreferences(MainActivity.PREFS_NAME,
MODE_PRIVATE)
        return prefs.getInt(MainActivity.KEY_USER_ID, 1)
    }
    override fun onDestroy() {
        dbHelper.close()
        super.onDestroy()
    }
}

```

Рисунок 46 – WatchActivity.kt

```

<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#FFFFFF">

    <LinearLayout
        android:id="@+id/main"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        tools:context=".AddEditActivity">

        <!-- Верхняя красная панель -->
        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="115dp"
            android:background="#FF0000"
            android:gravity="bottom"
            android:orientation="horizontal">

            <Button
                android:id="@+id/profileButton"
                android:layout_width="0dp"
                android:layout_height="wrap_content"
                android:layout_marginStart="5dp"
                android:layout_marginEnd="75dp"
                android:layout_marginBottom="8dp"
                android:layout_weight="1"
                android:backgroundTint="#FF0000"
                android:text="Профиль"
                android:textColor="#FFFFFF" />

            <Button
                android:id="@+id/btnBack"
                android:layout_width="0dp"
                android:layout_height="wrap_content"
                android:layout_marginStart="75dp"
                android:layout_marginEnd="15dp"

```

Рисунок 47 – activity_add_edit.xml

```

        android:layout_marginBottom="8dp"
        android:layout_weight="1"
        android:backgroundTint="#FF0000"
        android:text="Назад"
        android:textColor="#FFFFFF" />
</LinearLayout>

<!-- Заголовок -->
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="center"
    android:layout_marginTop="20dp"
    android:textSize="25dp"
    android:textStyle="bold"
    android:textColor="@color/black"
    android:text="Добавление/Редактирование" />

<!-- Фото товара -->
<ImageView
    android:id="@+id/productImage_ImageView"
    android:layout_width="match_parent"
    android:layout_height="250dp"
    android:layout_marginTop="20dp"
    android:scaleType="centerCrop"
    android:background="#E7E7E7"
    app:srcCompat="@drawable/hardware_cabinet" />

<!-- Кнопка добавить картинку -->
<Button
    android:id="@+id/btnAddImage"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="center"
    android:layout_marginTop="10dp"
    android:backgroundTint="#FF0000"
    android:text="Добавить картинку"
    android:textColor="#FFFFFF" />

```

Рисунок 48 – activity_add_edit.xml

```

<!-- Выбор заказчика -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:paddingHorizontal="20dp"
    android:paddingVertical="10dp">

    <TextView
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:textSize="17dp"
        android:text="Заказчик:"
        android:textColor="@color/black"
        android:textStyle="bold"/>

    <Spinner
        android:id="@+id/spinnerUser"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1.5"
        android:background="@android:drawable/btn_dropdown"
        android:spinnerMode="dropdown" />

</LinearLayout>

<!-- Выбор товара -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:paddingHorizontal="20dp"
    android:paddingVertical="10dp">

    <TextView
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:textSize="17dp"
        android:text="Товар:"

```

Рисунок 49 – activity_add_edit.xml

```

        android:textColor="@color/black"
        android:textStyle="bold"/>

        <Spinner
            android:id="@+id/spinnerProduct"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1.5"
            android:background="@android:drawable/btn_dropdown"
            android:spinnerMode="dropdown" />
    </LinearLayout>

    <!-- Выбор даты -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:paddingHorizontal="20dp"
        android:paddingVertical="10dp">

        <TextView
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:textSize="17dp"
            android:text="Дата заказа:"
            android:textColor="@color/black"
            android:textStyle="bold"/>

        <Button
            android:id="@+id/btnSelectDate"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1.5"
            android:text="Выбрать дату"
            android:backgroundTint="#FF0000"
            android:textColor="#FFFFFF"
            android:textSize="14dp"/>
    </LinearLayout>

```

Рисунок 50 – activity_add_edit.xml

```

<!-- Отображение выбранной даты -->
<TextView
    android:id="@+id/dateOrder_TextView"
    android:paddingHorizontal="20dp"
    android:paddingVertical="5dp"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textSize="15dp"
    android:text="Дата не выбрана"
    android:textColor="#666666"
    android:gravity="end"/>

<!-- Выбор статуса -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:paddingHorizontal="20dp"
    android:paddingVertical="10dp">

    <TextView
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:textSize="17dp"
        android:text="Статус:"
        android:textColor="@color/black"
        android:textStyle="bold"/>

    <Spinner
        android:id="@+id/spinnerStatus"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1.5"
        android:background="@android:drawable/btn_dropdown"
        android:spinnerMode="dropdown" />
</LinearLayout>

<!-- Автоматически подтягиваемые поля (только для просмотра) -->

<!-- Цена -->

```

Рисунок 51 – activity_add_edit.xml

```

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:paddingHorizontal="20dp"
    android:paddingVertical="10dp">

    <TextView
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:textSize="17dp"
        android:text="Цена:"
        android:textColor="@color/black"
        android:textStyle="bold"/>

    <TextView
        android:id="@+id/price_TextView"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:textSize="17dp"
        android:text="0 ₺"
        android:textColor="@color/black"
        android:gravity="end"/>
</LinearLayout>

<!-- Время изготовления -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:paddingHorizontal="20dp"
    android:paddingVertical="10dp">

    <TextView
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:textSize="17dp"
        android:text="Срок изготовления:"

```

Рисунок 52 – activity_add_edit.xml

```

        android:textColor="@color/black"
        android:textStyle="bold"/>

<TextView
    android:id="@+id/productionTime_TextView"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_weight="1"
    android:textSize="17dp"
    android:text="-"
    android:textColor="@color/black"
    android:gravity="end"/>
</LinearLayout>

<!-- Описание -->
<TextView
    android:paddingHorizontal="20dp"
    android:paddingVertical="10dp"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textSize="17dp"
    android:text="Описание:"
    android:textColor="@color/black"
    android:textStyle="bold"/>

<TextView
    android:id="@+id/productDiscription_TextView"
    android:paddingHorizontal="20dp"
    android:paddingVertical="10dp"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textSize="15dp"
    android:text="Описание товара"
    android:textColor="#666666"/>

<!-- Кнопка сохранения -->
<Button
    android:id="@+id/btnSave"
    android:layout_width="200dp"
    android:layout_height="wrap_content"
    android:layout_gravity="center"

```

Рисунок 53 – activity_add_edit.xml

```

        android:layout_marginTop="30dp"
        android:layout_marginBottom="30dp"
        android:backgroundTint="#FF0000"
        android:text="Сохранить заказ"
        android:textColor="#FFFFFF"
        android:textSize="17dp"
        android:textStyle="bold"/>

    </LinearLayout>
</ScrollView>

```

Рисунок 54 – activity_add_edit.xml

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="#FFFFFF"
    tools:context=".MainActivity">
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="115dp"
        android:orientation="horizontal"
        android:background="#FF0000"
        android:gravity="bottom">
    </LinearLayout>
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:layout_marginTop="170dp"
        android:textSize="25dp"
        android:textStyle="bold"
        android:textColor="@color/black"
        android:text="Авторизация" />
    <EditText
        android:id="@+id/user_login"

```

Рисунок 55 – activity_main.xml

```

        android:layout_width="220dp"
        android:layout_height="50dp"
        android:layout_marginTop="50dp"
        android:layout_gravity="center"
        android:ems="10"
        android:inputType="text"
        android:textColor="@color/black"
        android:hint="Логин"/>
    <EditText
        android:id="@+id/user_password"
        android:layout_width="220dp"
        android:layout_height="50dp"
        android:layout_marginTop="10dp"
        android:layout_gravity="center"
        android:ems="10"
        android:inputType="textPassword"
        android:textColor="@color/black"
        android:hint="Пароль"/>
    <Button
        android:id="@+id/button_auth"
        android:layout_width="220dp"
        android:layout_height="50dp"
        android:layout_gravity="center"
        android:textSize="17dp"
        android:textStyle="bold"
        android:layout_marginTop="70dp"
        android:backgroundTint="#FF0000"
        android:textColor="#FFFFFF"
        android:text="Войти"/>
    <TextView
        android:id="@+id/toReg"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:layout_marginTop="50dp"
        android:textSize="17dp"
        android:textColor="#FF0000"
        android:textStyle="bold"
        android:text="Регистрация" />
</LinearLayout>

```

Рисунок 56 – activity_main.xml

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#FFFFFF"
    android:orientation="vertical"
    tools:context=".ProductActivity">

    <!-- Верхняя красная панель -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="80dp"
        android:background="#FF0000"
        android:gravity="bottom"
        android:orientation="horizontal">
        <Button
            android:id="@+id/profileButton"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_marginStart="5dp"
            android:layout_marginEnd="75dp"
            android:layout_marginBottom="8dp"
            android:layout_weight="1"
            android:backgroundTint="#FF0000"
            android:text="Профиль"
            android:textColor="#FFFFFF" />
        <Button
            android:id="@+id/btnBack"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_marginStart="75dp"
            android:layout_marginEnd="15dp"
            android:layout_marginBottom="8dp"
            android:layout_weight="1"
            android:backgroundTint="#FF0000"
            android:text="Назад"
            android:textColor="#FFFFFF" />
    </LinearLayout>

```

Рисунок 57 – activity_product.xml

```

<!-- Заголовок -->
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="center"
    android:layout_marginTop="10dp"
    android:textSize="25dp"
    android:textStyle="bold"
    android:textColor="@color/black"
    android:text="Заказы" />

<!-- ♦ Панель поиска и фильтра -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:paddingHorizontal="20dp"
    android:paddingVertical="10dp"
    android:gravity="center_vertical">

    <!-- Поиск по названию -->
    <EditText
        android:id="@+id/searchEditText"
        android:layout_width="0dp"
        android:layout_height="40dp"
        android:layout_weight="2"
        android:hint="Поиск по названию..."
        android:textSize="14sp"
        android:paddingHorizontal="10dp"
        android:background="@android:drawable/edit_text"
        android:inputType="text"
        android:maxLines="1"/>

    <!-- Фильтр по статусу -->
    <Spinner
        android:id="@+id/statusFilterSpinner"
        android:layout_width="0dp"
        android:layout_height="40dp"
        android:layout_weight="1.5"
        android:layout_marginStart="10dp"
        android:background="@android:drawable/btn_dropdown"/>

```

Рисунок 58 – activity_product.xml

```

<!-- Кнопка очистки -->
<Button
    android:id="@+id/btnClearFilter"
    android:layout_width="wrap_content"
    android:layout_height="40dp"
    android:layout_marginStart="10dp"
    android:text="X"
    android:minWidth="40dp"
    android:backgroundTint="#FF0000"
    android:textColor="#FFFFFF"
    android:textSize="16sp"/>
</LinearLayout>

<!-- Список заказов -->
<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/Product_List"
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1"
    android:layout_marginHorizontal="20dp"
    android:layout_marginVertical="10dp"
    android:background="#E7E7E7"/>

<!-- Кнопка добавления -->
<Button
    android:id="@+id/AddButton"
    android:layout_width="200dp"
    android:layout_height="wrap_content"
    android:layout_gravity="center"
    android:layout_marginBottom="20dp"
    android:backgroundTint="#FF0000"
    android:text="+"
    android:textColor="#FFFFFF"
    android:textSize="25dp"/>
</LinearLayout>

```

Рисунок 59 – activity_product.xml

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="#FFFFFF"
    tools:context=".RegActivity">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="115dp"
        android:orientation="horizontal"
        android:background="#FF0000"
        android:gravity="bottom">

        <Button
            android:id="@+id/btnBack"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:layout_marginStart="275dp"
            android:layout_marginEnd="15dp"
            android:layout_marginBottom="8dp"
            android:backgroundTint="#FF0000"
            android:text="Назад"
            android:textColor="#FFFFFF" />
    </LinearLayout>

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:layout_marginTop="120dp"
        android:textSize="25dp"
        android:textStyle="bold"
        android:text="Регистрация"
        android:textColor="@color/black"/>

```

Рисунок 60 – activity_reg.xml

```
<EditText
    android:id="@+id/user_firstname"
    android:layout_width="220dp"
    android:layout_height="50dp"
    android:layout_marginTop="25dp"
    android:layout_gravity="center"
    android:ems="10"
    android:inputType="text"
    android:hint="Имя"
    android:textColor="@color/black"/>

<EditText
    android:id="@+id/user_lastname"
    android:layout_width="220dp"
    android:layout_height="50dp"
    android:layout_marginTop="10dp"
    android:layout_gravity="center"
    android:ems="10"
    android:inputType="text"
    android:hint="Фамилия"
    android:textColor="@color/black"/>

<EditText
    android:id="@+id/user_middlename"
    android:layout_width="220dp"
    android:layout_height="50dp"
    android:layout_marginTop="10dp"
    android:layout_gravity="center"
    android:ems="10"
    android:inputType="text"
    android:hint="Отчество"
    android:textColor="@color/black"/>

<EditText
    android:id="@+id/user_login"
    android:layout_width="220dp"
    android:layout_height="50dp"
    android:layout_marginTop="10dp"
    android:layout_gravity="center"
    android:ems="10"
    android:inputType="text"
    android:hint="Логин"
    android:textColor="@color/black"/>
```

Рисунок 61 – activity_reg.xml

```
<EditText
    android:id="@+id/user_password"
    android:layout_width="220dp"
    android:layout_height="50dp"
    android:layout_marginTop="10dp"
    android:layout_gravity="center"
    android:ems="10"
    android:inputType="textPassword"
    android:hint="Пароль"
    android:textColor="@color/black"/>
<Button
    android:id="@+id/button_reg"
    android:layout_width="250dp"
    android:layout_height="50dp"
    android:layout_gravity="center"
    android:textSize="17dp"
    android:layout_marginTop="50dp"
    android:backgroundTint="#FF0000"
    android:textColor="#FFFFFF"
    android:text="Зарегистрироваться"/>
</LinearLayout>
```

Рисунок 62 – activity_reg.xml

```

<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#FFFFFF">

    <LinearLayout
        android:id="@+id/main"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        tools:context=".UserProfileActivity">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="115dp"
            android:background="#FF0000"
            android:gravity="bottom"
            android:orientation="horizontal">

            <Button
                android:id="@+id/profileButton"
                android:layout_width="0dp"
                android:layout_height="wrap_content"
                android:layout_marginStart="5dp"
                android:layout_marginEnd="75dp"
                android:layout_marginBottom="8dp"
                android:layout_weight="1"
                android:backgroundTint="#FF0000"
                android:text="Профиль"
                android:textColor="#FFFFFF" />

            <Button
                android:id="@+id/btnBack"
                android:layout_width="0dp"
                android:layout_height="wrap_content"
                android:layout_marginStart="75dp"
                android:layout_marginEnd="15dp"
                android:layout_marginBottom="8dp"

```

Рисунок 63 – activity_user_profil.xml

```

        android:layout_weight="1"
        android:backgroundTint="#FF0000"
        android:text="Назад"
        android:textColor="#FFFFFF" />
</LinearLayout>

<!-- Заголовок -->
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="center"
    android:layout_marginTop="20dp"
    android:textSize="25dp"
    android:textStyle="bold"
    android:textColor="@color/black"
    android:text="Профиль пользователя" />

<!-- Фото пользователя -->
<ImageView
    android:id="@+id/userPhoto_ImageView"
    android:layout_width="250dp"
    android:layout_height="250dp"
    android:layout_marginTop="20dp"
    android:scaleType="centerCrop"
    android:background="#E7E7E7"
    android:layout_gravity="center"
    app:srcCompat="@drawable/profil_image" />

<!-- Кнопка добавить картинку -->
<Button
    android:id="@+id/btnAddImage"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="center"
    android:layout_marginTop="10dp"
    android:backgroundTint="#FF0000"
    android:text="Добавить картинку"
    android:textColor="#FFFFFF" />

```

Рисунок 63 – activity_reg.xml

```

<!-- Фамилия (редактируемое) -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:paddingHorizontal="20dp"
    android:paddingVertical="10dp">

    <TextView
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:textSize="17dp"
        android:text="Фамилия:"
        android:textColor="@color/black"
        android:textStyle="bold"/>

    <EditText
        android:id="@+id/userLastname_EditText"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1.5"
        android:textSize="17dp"
        android:textColor="@color/black"
        android:inputType="textPersonName"/>
</LinearLayout>

<!-- Имя (редактируемое) -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:paddingHorizontal="20dp"
    android:paddingVertical="10dp">

    <TextView
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:textSize="17dp"
        android:text="Имя:"

```

Рисунок 64 – activity_reg.xml

```

        android:textColor="@color/black"
        android:textStyle="bold"/>

<EditText
    android:id="@+id/userFirstname_EditText"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_weight="1.5"
    android:textSize="17dp"
    android:textColor="@color/black"
    android:inputType="textPersonName"/>
</LinearLayout>

<!-- Отчество (редактируемое) -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:paddingHorizontal="20dp"
    android:paddingVertical="10dp">

    <TextView
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:textSize="17dp"
        android:text="Отчество:"
        android:textColor="@color/black"
        android:textStyle="bold"/>

    <EditText
        android:id="@+id/userMiddlename_EditText"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1.5"
        android:textSize="17dp"
        android:textColor="@color/black"
        android:inputType="textPersonName"/>
</LinearLayout>

```

Рисунок 65 – activity_reg.xml

```

<!-- Роль (только для чтения) -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:paddingHorizontal="20dp"
    android:paddingVertical="10dp">

    <TextView
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:textSize="17dp"
        android:text="Роль:"
        android:textColor="@color/black"
        android:textStyle="bold"/>

    <TextView
        android:id="@+id/userRole_TextView"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1.5"
        android:textSize="17dp"
        android:textColor="#666666"/>
</LinearLayout>

<!-- Логин (только для чтения) -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:paddingHorizontal="20dp"
    android:paddingVertical="10dp">
    <TextView
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:textSize="17dp"
        android:text="Логин:"
        android:textColor="@color/black"
        android:textStyle="bold"/>

```

Рисунок 66 – activity_reg.xml

```

        <TextView
            android:id="@+id/userLogin_TextView"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1.5"
            android:textSize="17dp"
            android:textColor="#666666"/>
    </LinearLayout>

    <!-- Пароль (только для чтения) -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:paddingHorizontal="20dp"
        android:paddingVertical="10dp">

        <TextView
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:textSize="17dp"
            android:text="Пароль:"
            android:textColor="@color/black"
            android:textStyle="bold"/>

        <TextView
            android:id="@+id/userPassword_TextView"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1.5"
            android:textSize="17dp"
            android:textColor="#666666"/>
    </LinearLayout>

```

Рисунок 67 – activity_reg.xml

```

        <!-- Кнопка сохранения -->
        <Button
            android:id="@+id/btnSave"
            android:layout_width="200dp"
            android:layout_height="wrap_content"
            android:layout_gravity="center"
            android:layout_marginTop="30dp"
            android:layout_marginBottom="30dp"
            android:backgroundTint="#FF0000"
            android:text="Сохранить изменения"
            android:textColor="#FFFFFF"
            android:textSize="17dp"
            android:textStyle="bold"/>

    </LinearLayout>
</ScrollView>

```

Рисунок 68 – activity_reg.xml

```

<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    >

    <LinearLayout
        xmlns:app="http://schemas.android.com/apk/res-auto"
        xmlns:tools="http://schemas.android.com/tools"
        android:id="@+id/main"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:background="#FFFFFF"
        tools:context=".WatchActivity">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="115dp"
            android:background="#FF0000"
            android:gravity="bottom"
            android:orientation="horizontal">

```

Рисунок 69 – activity_watch.xml

```

        <Button
            android:id="@+id/profileButton"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_marginStart="5dp"
            android:layout_marginEnd="75dp"
            android:layout_marginBottom="8dp"
            android:layout_weight="1"
            android:backgroundTint="#FF0000"
            android:text="Профиль"
            android:textColor="#FFFFFF" />

        <Button
            android:id="@+id/btnBack"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_marginStart="75dp"
            android:layout_marginEnd="15dp"
            android:layout_marginBottom="8dp"
            android:layout_weight="1"
            android:backgroundTint="#FF0000"
            android:text="Назад"
            android:textColor="#FFFFFF" />
    </LinearLayout>

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:layout_marginTop="20dp"
        android:textSize="25dp"
        android:textStyle="bold"
        android:textColor="@color/black"
        android:text="Просмотр" />

    //Фото
    <ImageView
        android:id="@+id/productImage_ImageView"
        android:layout_width="match_parent"
        android:layout_height="250dp"
        android:layout_marginTop="20dp"

```

Рисунок 70 – activity_watch.xml

```

        android:scaleType="centerCrop"
        app:srcCompat="@drawable/hardware_cabinet" />
//Название
<TextView
    android:id="@+id/productName_TextView"
    android:paddingHorizontal="20dp"
    android:paddingVertical="10dp"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textStyle="bold"
    android:textSize="20dp"
    android:text="TextView"
    android:textColor="@color/black"/>

//Заказчик
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal">

    <TextView
        android:paddingHorizontal="50dp"
        android:paddingVertical="10dp"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:textSize="17dp"
        android:text="Заказчик:"
        android:textColor="@color/black"/>

    <TextView
        android:id="@+id/userName_TextView"
        android:paddingHorizontal="20dp"
        android:paddingVertical="10dp"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:textSize="17dp"
        android:text="TextView"
        android:textColor="@color/black"/>
</LinearLayout>

```

Рисунок 71 – activity_watch.xml

```

//Дата заказа
<TextView
    android:id="@+id/dateOrder_TextView"
    android:paddingHorizontal="20dp"
    android:paddingVertical="10dp"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textSize="17dp"
    android:text="TextView"
    android:textColor="@color/black"/>

//Время изготовления
<TextView
    android:id="@+id/productionTime_TextView"
    android:paddingHorizontal="20dp"
    android:paddingVertical="10dp"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textSize="17dp"
    android:text="TextView"
    android:textColor="@color/black"/>

//Статус
<TextView
    android:id="@+id/productStatus_TextView"
    android:paddingHorizontal="20dp"
    android:paddingVertical="10dp"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textSize="17dp"
    android:text="TextView"
    android:textColor="@color/black"/>

//Описание
<TextView
    android:id="@+id/productDiscription_TextView"
    android:paddingHorizontal="20dp"
    android:paddingVertical="10dp"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textSize="17dp"
    android:text="TextView"
    android:textColor="@color/black"/>

```

Рисунок 72 – activity_watch.xml

```

//Цена
<TextView
    android:id="@+id/price_TextView"
    android:paddingHorizontal="20dp"
    android:paddingVertical="10dp"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:gravity="end"
    android:textStyle="bold"
    android:textSize="20dp"
    android:text="TextView"
    android:textColor="@color/black"/>

<Button
    android:id="@+id/red_button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Редактировать"
    android:layout_gravity="center"
    android:backgroundTint="#FF0000"
    android:textColor="#FFFFFF"
    android:layout_marginBottom="150dp"/>

</LinearLayout>
</ScrollView>

```

Рисунок 73 – activity_watch.xml

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:paddingBottom="20dp"
    android:background="#E7E7E7"
    android:layout_marginBottom="20dp"
    android:layout_marginTop="20dp"
    android:paddingHorizontal="20dp"
    android:clickable="true"
    android:focusable="true">

```

Рисунок 74 – activity_in_list.xml

```

<ImageView
    android:id="@+id/product_list_image"
    android:layout_width="match_parent"
    android:layout_height="250dp"
    android:scaleType="centerCrop"
    android:contentDescription="@string/app_name"
    app:srcCompat="@drawable/hardware_cabinet" />

<TextView
    android:id="@+id/product_list_name"
    android:paddingHorizontal="20dp"
    android:paddingVertical="10dp"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textStyle="bold"
    android:textSize="20sp"
    android:text="Название товара"
    android:textColor="@color/black"/>

<TextView
    android:id="@+id/product_list_status"
    android:paddingHorizontal="20dp"
    android:paddingVertical="10dp"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textSize="17sp"
    android:text="Статус"
    android:textColor="@color/black"/>

<TextView
    android:id="@+id/product_list_user"
    android:paddingHorizontal="20dp"
    android:paddingVertical="10dp"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textSize="17sp"
    android:text="ФИО пользователя"
    android:textColor="@color/black"/>

```

Рисунок 75 – activity_in_list.xml

```

<TextView
    android:id="@+id/product_list_date"
    android:paddingHorizontal="20dp"
    android:paddingVertical="10dp"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textSize="17sp"
    android:text="Дата заказа"
    android:textColor="@color/black"/>

<TextView
    android:id="@+id/product_list_price"
    android:paddingHorizontal="20dp"
    android:paddingVertical="10dp"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:gravity="end"
    android:textStyle="bold"
    android:textSize="20sp"
    android:text="0 ₺"
    android:textColor="@color/black"/>

<Button
    android:id="@+id/product_list_button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Подробнее"
    android:layout_gravity="center"
    android:backgroundTint="#FF0000"
    android:textColor="#FFFFFF"/>
</LinearLayout>

```

Рисунок 75 – activity_in_list.xml